TP MATH INFORMATIQUE

Tonin PACHER

March 24, 2026

1 Matrice 2D

Toutes les matrices aléatoires seront de taille (8,16).
On peut donc déjà définir les bornes maximales pour les différentes entropies :

$$\max(H(X)) = \log 2(8) \ = 3$$
$$\max(H(Y)) = \log 2(16) \ = 4$$
$$\max(H(X,Y)) = \log 2(128) \ = 7$$
$$\max(I(X;Y)) = \min(H(X), H(Y))$$

1.1 Codage de l'Entropie H(X)

Première étape : nous allons coder l'entropie classique se basant sur une seule variable aléatoire, ici H(X).
Rappelons la formule :

$$\sum_{n=1}^{N} p(xi) \times \log 2(p(xi))$$

Le code Python est donc :

```python
def Entropie(Z):
    """ entropie de H(X)"""
    Z = Z + eps
    Z = Z / Z.sum()
    Px = np.sum(Z, axis=1)
    return -np.sum(Px * np.log2(Px))
```

Après exécution sur une matrice 2D, cela nous donne :

[[459 526 195 684 501 754 763 479 319 798 850 525 874 363 785 319] [321 261 491 466 320 8 762 932 49 73 964 111 565 87 822 347] [584 885

472 564 862 425 822 368 519 823 522 609 910 161 255 507] [137 825 636 726 215 927 103 572 95 27 193 173 573 840 792 880] [992 585 581 573 417 143 539 221 608 309 754 724 43 953 608 108] [517 381 899 727 261 823 518 418 195 272 1 65 442 988 18 589] [140 914 927 958 35 311 438 35 180 494 568 298 594 330 197 179] [40 960 156 417 6 818 6 611 77 231 947 239 899 935 109 513]
nous donne : 2.9875584206597683
La borne maximale notée plus haut est 3, c'est donc cohérent étant donné qu'on est sur un tirage aléatoire : l'incertitude est très élevée.
Après test sur un Dirac, on obtient bien 0.

1.2 Codage de l'Entropie jointe H(X,Y)

Deuxième étape : nous allons coder l'entropie jointe. On rappelle la formule :

$$\sum_{n=1}^{N} p(xi, yj) \times \log 2(p(xi, yj)$$

Le code Python est donc :

```python
def Entropie_jointe(Z):
    """ entropie jointe entre H(X) et H(Y) """
    Z = Z + eps
    P_Z = Z / np.sum(np.ndarray.flatten(Z))
    P_flat = P_Z.flatten()
    return -1 * np.sum((P_flat) * np.log2(P_flat))
```

Après exécution sur une matrice 2D, cela nous donne :

[[981 817 354 961 359 887 905 787 486 567 940 269 150 824 465 2] [4 41 242 261 204 554 413 635 14 224 127 553 550 915 683 878] [559 72 497 686 778 31 534 309 591 78 386 561 118 523 150 702] [274 597 849 847 827 935 441 635 307 470 72 398 352 400 186 304] [65 495 78 15 563 768 803 729 318 909 876 753 668 525 428 471] [530 510 125 277 322 405 244 221 436 345 577 998 756 623 343 453] [126 632 693 653 637 440 197 101 456 376 229 398 319 692 222 620] [620 786 701 916 901 574 580 972 96 335 463 265 390 831 682 924]
nous donne : 6.746527033542511
C'est cohérent car la borne maximale notée plus haut est 7, et comme on fait un tirage aléatoire on se rapproche de la borne maximale car l'incertitude est élevée.

1.3 Codage de l'Entropie conditionnelle H(X| Y)

Pour coder l'entropie conditionnelle, nous allons utiliser les formules rédigées au-dessus. On rappelle la formule de l'entropie conditionnelle :

$$H(X,Y) - H(Y)$$

Le code Python est donc :

```python
def Entropie_conditionnel(Z):
    """ calcul de l'entropie conditionnel H(X | Y)"""
    J = Entropie_jointe(Z)
    E = Entropie_Y(Z)
    return J - E
```

Il faut prendre en compte que la fonction Entropie Y existe ; il faut juste changer la valeur de `axis` dans l'entropie de X.
Après exécution sur une matrice 2D, cela nous donne :

[[981 817 354 961 359 887 905 787 486 567 940 269 150 824 465 2] [4 41 242 261 204 554 413 635 14 224 127 553 550 915 683 878] [559 72 497 686 778 31 534 309 591 78 386 561 118 523 150 702] [274 597 849 847 827 935 441 635 307 470 72 398 352 400 186 304] [65 495 78 15 563 768 803 729 318 909 876 753 668 525 428 471] [530 510 125 277 322 405 244 221 436 345 577 998 756 623 343 453] [126 632 693 653 637 440 197 101 456 376 229 398 319 692 222 620] [620 786 701 916 901 574 580 972 96 335 463 265 390 831 682 924]
nous donne : 3.7670756143714543

1.4 Codage de l'information mutuelle I(X;Y)

Pour coder l'information mutuelle de X et Y, nous allons utiliser les formules rédigées à l'avance pour des questions de facilité. On rappelle que la formule est donc :

$$H(X) + H(Y) - H(X,Y)$$

Le code Python est donc :

```python
def Information_Mutuelle(Z):
    """ calcul de l'information mutuelle I(X;Y) """
    J = Entropie_jointe(Z)
    Ex = Entropie(Z)
```

```
    Ey = Entropie_Y(Z)
    return Ex + Ey - J
```

Après exécution sur une matrice 2D, cela nous donne :

[[981 817 354 961 359 887 905 787 486 567 940 269 150 824 465 2] [4 41 242 261 204 554 413 635 14 224 127 553 550 915 683 878] [559 72 497 686 778 31 534 309 591 78 386 561 118 523 150 702] [274 597 849 847 827 935 441 635 307 470 72 398 352 400 186 304] [65 495 78 15 563 768 803 729 318 909 876 753 668 525 428 471] [530 510 125 277 322 405 244 221 436 345 577 998 756 623 343 453] [126 632 693 653 637 440 197 101 456 376 229 398 319 692 222 620] [620 786 701 916 901 574 580 972 96 335 463 265 390 831 682 924]
nous donne : 0.21092405738769138
On sait que la borne maximale de l'information mutuelle est la plus petite valeur entre H(X) et H(Y). Après calcul, on remarque que la valeur est très basse ; on peut donc en conclure que les informations X et Y n'ont pas beaucoup de lien.

2 Matrice 3D

Nous serons tout au long sur une matrice 3D de taille (8,16,7).
On peut donc définir les bornes maximales :

$$\max(H(A)) = \log 2(8) \ = 3$$
$$\max(H(B)) = \log 2(16) \ = 4$$
$$\max(H(C)) = \log 2(7) \ = 2.807$$
$$\max(H(X,Y)) = \log 2(128) \ = 7$$
$$\max(H(A,B,C)) = \log 2(896) \ = 9.8$$
$$\max(H(A,B|C)) = \log 2(128) \ = 7$$

2.1 Codage de l'entropie de la dernière variable 3D

Pour calculer l'entropie de la 3D, il faut faire la somme de l'axe 1 puis, avec ces résultats, re-sommer sur l'axe 0 afin d'obtenir le total de la 3D. Attention à bien renormaliser après calcul.

$$H = -\sum_{c=0}^{C-1} \left(\frac{\sum_{a=0}^{A-1} \sum_{b=0}^{B-1} \left(Z_1[a,b,c] + \varepsilon \right)}{\sum_{a=0}^{A-1} \sum_{b=0}^{B-1} \sum_{c=0}^{C-1} \left(Z_1[a,b,c] + \varepsilon \right)} \right) \log_2 \left(\frac{\sum_{a=0}^{A-1} \sum_{b=0}^{B-1} \left(Z_1[a,b,c] + \varepsilon \right)}{\sum_{a=0}^{A-1} \sum_{b=0}^{B-1} \sum_{c=0}^{C-1} \left(Z_1[a,b,c] + \varepsilon \right)} \right)$$

La fonction est un peu différente de l'entropie 2D car on compresse les axes

que l'on ne veut pas. Étant donné qu'ici on est en 3D, il y a deux côtés à supprimer : l'axe 0 et l'axe 1.
Le code Python est donc :

```
def Entropie(Z): #calcul entropie peut importe la dimension du cube
    Z = np.array(Z, dtype=float) + eps
    Z = Z / Z.sum()
    Px = np.sum(Z, axis=1)
    return -np.sum(Px * np.log2(Px))
```

Je ne mettrai pas les exemples car les matrices sont trop grandes, cela prendrait trop de place. Le résultat obtenu sur une matrice générée aléatoirement est :
2.806586742493664
On remarque qu'on est très proche de la borne maximale ; on a donc une incertitude qui est maximale.

2.2 Codage de l'entropie conditionnelle jointe de H(A , B | C)

Pour calculer cela, on va utiliser des fonctions écrites auparavant. La formule est :

$$H(A, B \mid C) = H(A, B, C) - H(C)$$

Le code Python est donc :

```
def Entropie_jointe(Z): #calcul de l'entropie (X,Y)
    Z = np.array(Z, dtype=float) + eps
    P_Z = Z / np.sum(Z)
    return -np.sum(P_Z.flatten() * np.log2(P_Z.flatten()))
```

On applique simplement la formule avec les fonctions codées auparavant et on obtient : **6.7098072396548885**
On a donc un résultat très proche du maximum ; on peut donc en déduire que A et B dépendent très peu de C.

2.3 Codage de l'information mutuelle I(A,B;C)

Pour cela, on aura besoin de la fonction H(A,B) :

```
def Entropie_jointe2D(Z1, axis):
    """ Entropie jointe de deux variables
    en marginalisant la troisieme """
    Z1 = Z1 + eps
    Z1 = Z1 / Z1.sum()
```

```
    P_2D = np.sum(Z1, axis=axis)
    P_2D = P_2D / np.sum(P_2D)
    P_flat = P_2D.flatten()
    return -np.sum(P_flat * np.log2(P_flat))
```

Ensuite on applique la formule :

$$H(A,B) + H(C) - H(A,B,C)$$

Le code Python est donc :

```
def Information_Mutuelle(Z): #calcul info mutuelle avec formule
    return Entropie(Z) + Entropie_Y(Z) - Entropie_jointe(Z)
```

On obtient : **0.24095957591253025**

3 Hypercube

3.1 Codage de l'information mutuelle dirigée

La fonction `info_dirigee` calcule la somme des informations mutuelles conditionnelles de la forme :

$$\mathrm{ID}(W) = \sum_{t=1}^{N-1} I\big(X_t; X_{t+1} \mid X_1, \ldots, X_{t-1}\big)$$

où $I(A; B \mid C)$ représente l'information mutuelle conditionnelle entre A et B sachant C, et N est le nombre de pas temporels (ici $N = 7$). **Après developement de la formule nous obtenons :**

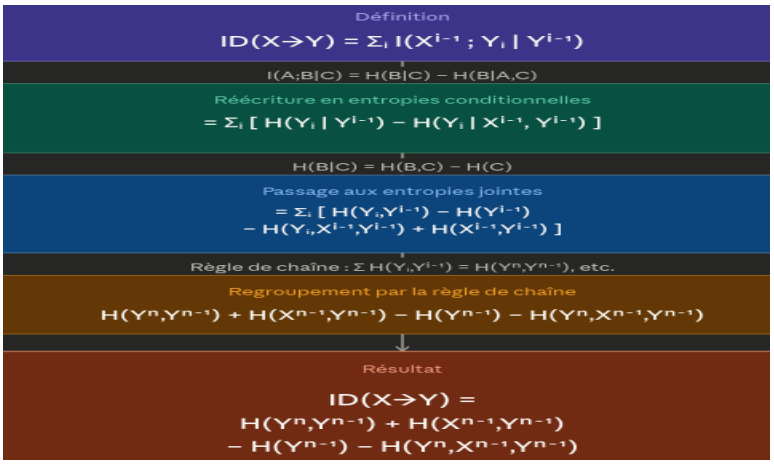

Figure 1: developement ID

3.2 Codage de l'information dirigée sur la matrice 3D

où $I(A; B \mid C)$ est l'information mutuelle conditionnelle, et N est le nombre de pas temporels de la matrice.

Le code Python est donc :

```
def info_dirigee(W):
    n = 8
    T = W.shape[1]
    C = np.zeros((n, n, n)) + eps #creation de la matrice vide remplis de 0
    for t in range(1, T):
        yt = np.argmax(W[1, t,   :])#recuperation des donnees de Y au temps
        xp = np.argmax(W[0, t-1, :])#recuperation des donnees de X au temps
        yp = np.argmax(W[1, t-1, :])#recuperation des donnees de Y au temps
        C[yt, xp, yp] += 1 #ajout dans ma matrice C
    #arrivee ici le tenseur est rempli
    C_yt_yp = C.sum(axis=1) #je somme les donnees de Y au temps t -1
    C_xp_yp = C.sum(axis=0) #je somme les donnees de X au temps t -1
    C_yp    = C_yt_yp.sum(axis=0) #je somme les donnees au temps Y present
    id_score = (Entropie_jointe(C_yt_yp) + Entropie_jointe(C_xp_yp)
               - Entropie_jointe(C_yp)   - Entropie_jointe(C))
    return max(id_score, 0.0)
```

Après exécution sur une matrice 3D générée aléatoirement :

```
W = np.random.random((8,8,20))
resultat = info_dirigee(W)
print("R sultat info dirigee :", resultat)
```

Résultat obtenu : 1.64

Analyse : La valeur positive indique que les vecteurs successifs de la matrice partagent une certaine dépendance temporelle. Plus la valeur est élevée, plus l'information est dirigée entre les étapes temporelles.

4 Étude de données réelles sur des cachalots

Cette partie vise à identifier l'individu Alpha au sein d'un groupe de 6 cachalots (Cesar, Fanny, Marius, Norine, Felix, Honoré) en utilisant l'Information Dirigée sur des données de trajectoires réelles.

Attention : dans la quantification, il y a un lien entre les valeurs numériques et les probabilités des valeurs de codage.

4.1 Récupération des données

Dans un premier temps, nous allons récupérer les données des fichiers CSV et stocker les données qui nous intéressent dans un dictionnaire que l'on

appellera DATA.

```
fichiers = {
    "Cesar":  "Trajectoire_Cesar_sec.csv",
    "Fanny":  "Trajectoire_Fanny_sec.csv",
    "Marius": "Trajectoire_Marius_sec.csv",
    "Norine": "Trajectoire_Norine_sec.csv",
    "Felix":  "Trajectoire_Felix_sec.csv",
    "Honore": "Trajectoire_Honore_sec.csv"
}
data = {}
for nom, f in fichiers.items():
    chemin = os.path.join(DOSSIER, f)
    try:
        df = pd.read_csv(chemin, header=None)
        data[nom] = {
            'long': df[1].values,
            'lat':  df[2].values,
            'prof': df[3].values,
            'sec':  df[5].values
        }
        print(f"OK : {nom}")
    except Exception as e:
        print(f"Fichier manquant : {chemin} ({e})")

noms = list(data.keys())
N = len(noms)
print(f"\n{N} animaux charges : {noms}\n")

if N < 2:
    raise RuntimeError("Pas assez de fichiers charges.")
```

4.2 Mise en place de la synchronisation

Dans cette partie, nous allons synchroniser les données. Le problème majeur est que les instants d'enregistrement ne sont pas alignés entre les animaux : chaque balise GPS émet à ses propres instants, indépendamment des autres. Afin de calculer l'Information Dirigée entre chaque paire, tous les animaux doivent partager exactement les mêmes instants de mesure.

Nous écrivons pour cela deux fonctions. La première, `fit_traj`, ajuste un polynôme de degré D sur les données brutes de chaque animal et l'évalue sur une grille temporelle régulière et commune à la paire, produisant une trajectoire lissée et synchronisée. La seconde, `traj_to_W_onehot`, convertit ces trajectoires en un tenseur W de vecteurs one-hot en encodant chaque déplacement comme l'un des 8 états directionnels définis précédemment.

Listing 1: Lissage polynomial et synchronisation

```python
t_min  = max(d['sec'].min() for d in data.values())
t_max  = min(d['sec'].max() for d in data.values())
t_sync = np.arange(t_min, t_max, 60)
t_norm = (t_sync - t_min) / (t_max - t_min)


for nom, d in data.items():
    print(f"{nom:8s} : debut={d['sec'].min():.0f}s  fin={d['sec'].max():.0f}s duree={(d['sec'].max()-d['sec'].min())/3600:.2f}h")


def fit_traj(nom, Ndeg, t_cible_norm, t0, t1):
    dat = data[nom]
    t_base_norm = (dat['sec'] - t0) / (t1 - t0)
    px = np.polyval(np.polyfit(t_base_norm, dat['long'], Ndeg), t_cible_norm
    py = np.polyval(np.polyfit(t_base_norm, dat['lat'],  Ndeg), t_cible_norm
    pz = np.polyval(np.polyfit(t_base_norm, dat['prof'], Ndeg), t_cible_norm
    return np.stack([px, py, pz], axis=1)
```

Au retour de cette fonction, les données de chaque animal se trouvent dans une matrice lissée ; il n'y a donc plus de problème de synchronisation pour l'animal.

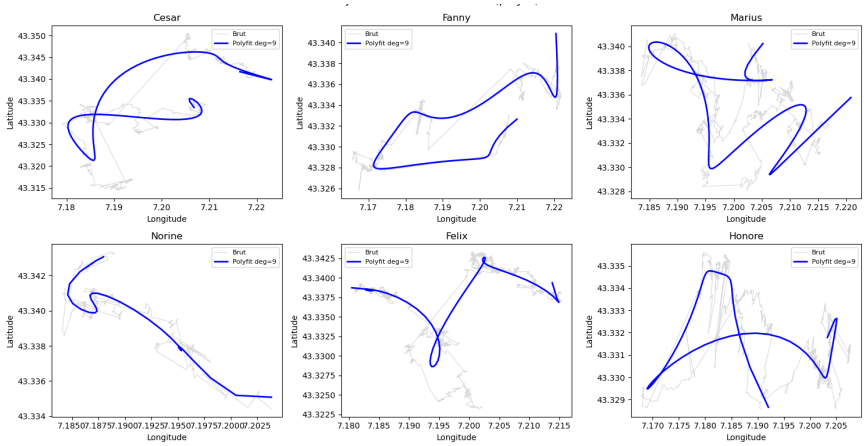

Figure 2: données apres lissage VS avant

Pour transformer les trajectoires continues en flux d'information discrets, nous construisons un tenseur W de dimension $(N \times T \times 8)$. L'état de chaque individu à l'instant t est codé en **binaire one-hot** sur 8 états : un vecteur de 8 cases avec un **1 à la direction la plus proche** (arrondi au cardinal le plus proche) et **0 partout ailleurs**.

Nous utilisons les variables : NM ND EM ED OM OD SM SD où M et D désignent respectivement la montée et la descente, et N S E O représentent les points cardinaux. Le lissage polynomial des trajectoires brutes est réalisé avec un degré $D = 9$, les temps étant normalisés sur $[0, 1]$ pour garantir la stabilité numérique.

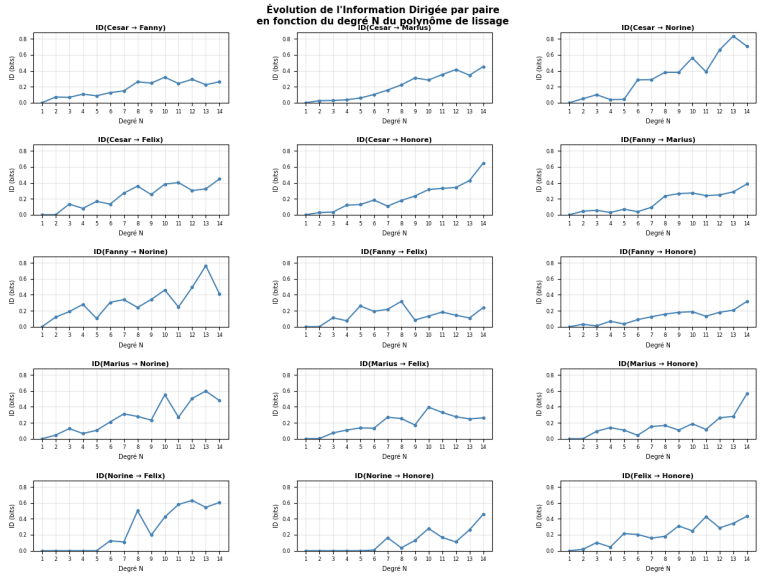

Figure 3: Explication du choix de degres 9

Listing 2: Création du tenseur

```
def traj_to_W_onehot(traj_dict, noms_list):
    N_loc = len(noms_list)
    T_loc = len(next(iter(traj_dict.values()))) - 1
    W = np.zeros((N_loc, T_loc, 8), dtype=int)
    for i, nom in enumerate(noms_list):
        traj = traj_dict[nom]
        dx = np.diff(traj[:, 0])
        dy = np.diff(traj[:, 1])
        dz = np.diff(traj[:, 2])
        angles = np.arctan2(dy, dx)

        card = np.full(T_loc, -1, dtype=int)
        card[(angles >= -np.pi/4) & (angles <   np.pi/4)] = 0
        card[(angles >=  np.pi/4) & (angles <  3*np.pi/4)] = 1
        card[(angles <  -np.pi/4) & (angles >= -3*np.pi/4)] = 2
        card[(angles >= 3*np.pi/4) | (angles <  -3*np.pi/4)] = 3
        assert np.all(card >= 0), f"Angle non couvert detecte pour {nom}"

```

```
        montee = (dz < 0).astype(int)
        etat   = card * 2 + montee
        for t in range(T_loc):
            W[i, t, etat[t]] = 1
    return W
```

Suite à cette fonction, nous obtenons un tenseur W.

Observons la différence des données avant et après lissage sur la profondeur pour mieux comprendre ce qui se passe.
C'est ici que notre premier problème se pose — nous allons le détailler.

Les données étant réelles, les enregistrements ne sont pas synchronisés, et chaque animal possède une durée d'émission différente. Si l'on souhaite travailler uniquement sur la fenêtre temporelle où les six animaux émettent simultanément, le tenseur obtenu en sortie est le suivant :

$$W \in \mathbb{R}^{6\times 24\times 8} \quad \Longrightarrow \quad 6 \text{ animaux} \times 24 \text{ instants} \times 8 \text{ états}$$

Ce tenseur est trop petit pour permettre une analyse statistiquement fiable. Avec seulement 24 instants, il est impossible de tirer une conclusion robuste étant donné que l'on découpe notre tenseur en 7 parties suite à l'analyse des graphes du nombre de découpe qui nous montre une stabilisation lorsque N = 7 : c'est comme lancer une pièce de monnaie une seule fois et en déduire que le résultat sera toujours identique.

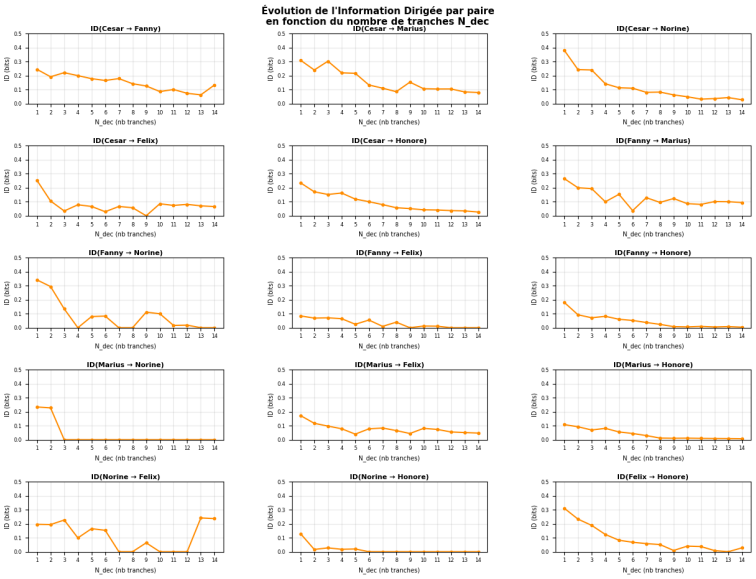

Figure 4: ID en fonction du nombre de découpe

Nous verrons par la suite comment interpréter ce résultat et quelles adaptations ont été apportées.

4.2.1 Visualisation du Tenseur W

Le tenseur W a une forme $(6 \times T \times 8)$. Pour chaque individu et chaque instant t, la ligne contient exactement un **1** à la position de l'état de déplacement et des **0** partout ailleurs. L'extrait ci-dessous montre l'indice de l'état actif (position du 1) pour chaque individu et un exemple de déplacement sur un animal dans le tenseur W. Le tenseur est prévu afin de pouvoir récupérer les temps $t-1$ nécessaires à l'application de l'information dirigée.

$$W = \begin{pmatrix} \text{Cesar} & [3,3,3,\ldots,2,2,2] \\ \text{Fanny} & [6,6,6,\ldots,6,6,6] \\ \text{Marius} & [2,2,2,\ldots,5,5,5] \\ \text{Norine} & [7,7,6,\ldots,6,6,6] \\ \text{Felix} & [3,3,3,\ldots,5,5,5] \\ \text{Honore} & [5,5,5,\ldots,1,1,1] \end{pmatrix}$$

Chaque ligne du tenseur W correspond à un animal, chaque colonne à un instant t (pas de 1 minute). La valeur entière en chaque case représente l'état directionnel actif parmi les 8 états possibles : 0 = E↓, 1 = E↑, 2 = N↓, 3 = N↑, 4 = S↓, 5 = S↑, 6 = O↓, 7 = O↑, où la lettre indique la direction cardinale du déplacement horizontal et la flèche le sens vertical (plongée/remontée).

4.3 Identification de l'Alpha par l'Information Dirigée

Nous pouvons revenir à notre problème :

Étant donné que notre tenseur W est trop petit et que certains animaux n'émettent pas longtemps, nous avons plusieurs solutions :
- **Solution 1 :** Calculer le temps d'émission seulement par couple pour maximiser le calcul de l'information mutuelle. C'est ce qui est réalisé dans le code ci-dessous.
- **Solution 2 :** Calculer l'information dirigée en supprimant du calcul les 3 animaux qui n'émettent pas assez, pour comparer les résultats.

Pour chaque paire d'animaux (i, j), une fenêtre temporelle commune $[t_0, t_1]$ est extraite, les trajectoires sont lissées et synchronisées sur une grille normalisée, puis l'information dirigée $\mathrm{ID}(i \to j)$ est calculée et stockée dans la matrice $M \in \mathbb{R}^{N \times N}$. Le score global de chaque animal est obtenu par

sommation des lignes : $s_i = \sum_{j \neq i} \mathrm{ID}(i \to j)$, et l'individu α est celui qui maximise s_i.

```
N_dec = 7  # nombre de tranches egales sur lesquels on moyenne l'ID

mat_global = np.zeros((N, N))
for i in range(N):
    for j in range(N):
        if i == j:
            continue
        t0 = max(data[noms[i]]['sec'].min(), data[noms[j]]['sec'].min())
        t1 = min(data[noms[i]]['sec'].max(), data[noms[j]]['sec'].max())
        t_norm_ij = (np.arange(t0, t1, 60) - t0) / (t1 - t0)
        traj_i = fit_traj(noms[i], N_optimal, t_norm_ij, t0, t1)
        traj_j = fit_traj(noms[j], N_optimal, t_norm_ij, t0, t1)
        W_ij   = traj_to_W_onehot({noms[i]: traj_i, noms[j]: traj_j}, [noms[
        T_ij     = W_ij.shape[1]
        taille   = T_ij // N_dec
        scores   = []
        for k in range(N_dec):
            debut  = k * taille
            fin    = debut + taille if k < N_dec - 1 else T_ij
            W_tr   = W_ij[:, debut:fin, :]
            if W_tr.shape[1] > 1:
                scores.append(info_dirigee(W_tr))
        mat_global[i, j] = np.mean(scores) if scores else 0.0

scores_global = mat_global.sum(axis=1)
alpha_global  = noms[np.argmax(scores_global)]
print("MATRICE ID (ligne = source, colonne = cible)")
print(pd.DataFrame(mat_global, index=noms, columns=noms).round(3))
```

On obtient donc en sortie notre alpha calculé.

4.3.1 Analyse des résultats numériques et démonstration des limites

Nous allons ici montrer les limites en montrant qu'en enlevant les 3 animaux qui émettent le moins on obtient un résultat différent.
Matrice ID avec les 6 animaux : Avec 6 animaux, le résultat serait Felix en tant qu'alpha. Or, quand on regarde les trajectoires de Felix, on voit que c'est l'un des animaux qui plonge le moins profondément ; c'est donc une erreur de résultat due au fait que les animaux qui n'émettent pas longtemps faussent les résultats.

Passons maintenant à 3 animaux, ceux qui émettent le plus longtemps :

Source ↓ / Cible →	**Cesar**	**Fanny**	**Marius**	**Norine**	**Felix**	**Honore**	**Score Alpha**
Cesar	-	0.262	0.223	0.381	0.359	0.179	1.404
Fanny	0.173	-	0.235	0.242	0.318	0.158	1.126
Marius	0.160	0.163	-	0.280	0.253	0.166	1.022
Norine	0.174	0.306	0.298	-	0.500	0.036	1.314
Felix	0.171	0.088	0.257	0.834	-	0.180	**1.530**
Honore	0.200	0.160	0.161	0.161	0.455	-	1.137

Table 1: Matrice d'Influence ID Globale (en bits), $N = 8$, codage one-hot 8 états pour 6 animaux

Source ↓ / Cible →	**Cesar**	**Fanny**	**Marius**	**Score Alpha**
Cesar	-	0.262	0.223	0.485
Fanny	0.173	-	0.235	0.408
Marius	0.160	0.163	-	0.323

Table 2: Matrice d'Influence ID Globale (en bits), $N = 8$, codage one-hot 8 états

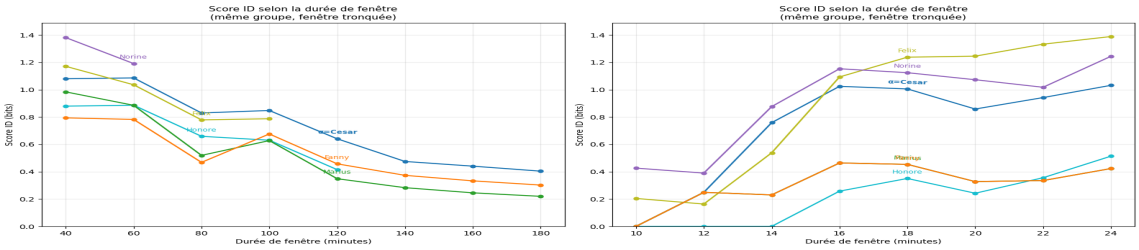

Figure 5: Différents scores ID en fonction du temps

Après analyse des résultats, nous trouvons Cesar comme alpha. Si nous regardons dans le détail, nous voyons que Cesar et Fanny sont majoritairement leaders, mais après moyenne, Fanny passe en deuxième position. Les résultats ne sont pas concluants étant donné que seulement 3 animaux émettent pendant un temps suffisamment long pour éviter l'aléatoire dans les résultats ; les scores sont donc faussés.

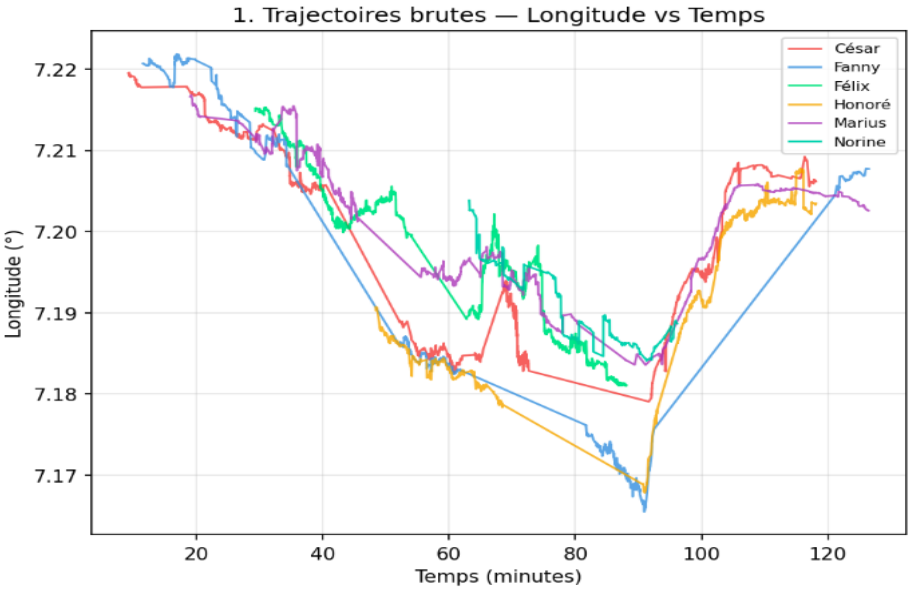

Figure 6: Déplacement en longitude

4.4 L'Égalité de Massey : Causalité vs Corrélation

Nous allons examiner l'égalité de Massey afin de vérifier si les résultats obtenus ne sont pas dus à une erreur de calcul :
$R = [ID(i \to j) + ID(j \to i)] - I(X;Y)$.

```
def _construire_C4(W):
    n = 8 #nombre de d coupe du cube
    T = W.shape[1] #on recupe la shape 1
    C4 = np.zeros((n, n, n, n)) + eps #creation cube rempli de 0
    for t in range(1, T):
        xt = np.argmax(W[0, t,   :]) #  tat  X pr sent  (temps t)
        yt = np.argmax(W[1, t,   :]) #  tat  Y pr sent  (temps t)
        xp = np.argmax(W[0, t-1, :]) #  tat  X pass     (temps  t 1 )
        yp = np.argmax(W[1, t-1, :]) #  tat  Y pass     (temps  t 1 )
        C4[xt, yt, xp, yp] += 1 #placement dans le cube
    return C4

def id_brut(W):
    C4 = _construire_C4(W)
    C        = C4.sum(axis=0) # cube C[yt, xp, yp]     marginale sur xt
    C_yt_yp  = C.sum(axis=1) # marginalise xp     jointe (yt, yp)
    C_xp_yp  = C.sum(axis=0) # marginalise yt     jointe (xp, yp)
    C_yp     = C_yt_yp.sum(axis=0) # marginalise yt     marginale yp
    return (Entropie_jointe(C_yt_yp) + Entropie_jointe(C_xp_yp)- Entropie_jo
- Entropie_jointe(C)) #application de la formule
```

```python


def im_massey(W):
    C4 = _construire_C4(W)
    C_yt_yp     = C4.sum(axis=(0, 2))# marginalise xt et xp -> (yt, yp)
    C_xt_xp     = C4.sum(axis=(1, 3))# marginalise yt et yp -> (xt, xp)
    C_xp_yp     = C4.sum(axis=(0, 1))# marginalise xt et yt -> (xp, yp)
    C_yp        = C_xp_yp.sum(axis=0)# marginalise xp -> (yp)
    C_xp        = C_xp_yp.sum(axis=1)# marginalise yp -> (xp)
    C_yt_xp_yp = C4.sum(axis=0)# marginalise xt -> cube de ID( X Y )
    C_xt_yp_xp = C4.sum(axis=1)# marginalise yt ->  cube de ID( Y X )
    return (Entropie_jointe(C_yt_yp)+ Entropie_jointe(C_xt_xp)+ 2 * Entropie

#cela marche car le calcul de l'id est bas  sur les temps t-1
for i in range(N):
    for j in range(i + 1, N):
        nom_a, nom_b = noms[i], noms[j]
        t0 = max(data[nom_a]['sec'].min(), data[nom_b]['sec'].min()) #recupe
        t1 = min(data[nom_a]['sec'].max(), data[nom_b]['sec'].max())
        t_norm_ij = (np.arange(t0, t1, 60) - t0) / (t1 - t0)
        traj_a = fit_traj(nom_a, N_optimal, t_norm_ij, t0, t1)
        traj_b = fit_traj(nom_b, N_optimal, t_norm_ij, t0, t1) #synchronisat
        W_ij   = traj_to_W_onehot({nom_a: traj_a, nom_b: traj_b}, [nom_a, no
        T_ij   = W_ij.shape[1]
        id_ab  = id_brut(W_ij) #application ID brut qui enleve le max 0
        id_ba  = id_brut(W_ij[[1, 0]]) #pareil
        somme  = id_ab + id_ba #recuperation res
        im_ref = im_massey(W_ij) #id de massey
        residu = somme - im_ref #formule
        label = f"{nom_a}   {nom_b}"
        print(f"{label:<20} {id_ab:>10.4f} {id_ba:>10.4f} {somme:>10.4f} {im

print("  R sidu = 0.0000 sur toutes les paires : identit  alg brique  v r
```

Pour le code de Massey, il n'est pas nécessaire d'adapter la puissance de Y dans la formule, étant donné que l'ID dirigé calcule déjà le temps $t-1$ dans un sens, ce qui supprime des étapes pour Massey. il faut par contre modifier legerement le code de l'ID pour évider les residu et agir sur les grandes valeurs en enlevant le max (...,0)

Paire	ID(A→B)	ID(B→A)	Somme	IM Massey	Résidu
Cesar → Fanny	0.2461	0.2126	0.4587	0.4587	-0.0000
Cesar → Marius	0.3106	0.1486	0.4592	0.4592	-0.0000
Cesar → Norine	0.3812	0.1145	0.4958	0.4958	-0.0000
Cesar → Felix	0.2533	0.2710	0.5242	0.5242	-0.0000
Cesar → Honore	0.2348	0.2377	0.4725	0.4725	-0.0000
Fanny → Marius	0.2650	0.1681	0.4331	0.4331	-0.0000
Fanny → Norine	0.3419	0.2091	0.5510	0.5510	-0.0000
Fanny → Felix	0.0843	0.1573	0.2416	0.2416	-0.0000
Fanny → Honore	0.1805	0.2181	0.3986	0.3986	-0.0000
Marius → Norine	0.2340	0.1822	0.4162	0.4162	-0.0000
Marius → Felix	0.1722	0.1389	0.3111	0.3111	-0.0000
Marius → Honore	0.1080	0.2307	0.3387	0.3387	-0.0000
Norine → Felix	0.1959	0.7352	0.9311	0.9311	0.0000
Norine → Honore	0.1283	0.3347	0.4630	0.4630	-0.0000
Felix → Honore	0.3102	0.1955	0.5057	0.5057	0.0000

Sur l'ensemble des 15 paires, le résidu entre la somme $ID(A \to B) + ID(B \to A)$ et la valeur théorique de Massey est nul à la précision numérique. L'identité algébrique est donc vérifiée, ce qui valide l'implémentation du calcul d'information dirigée.

On peut également noter que le résidu est systématiquement nul quelle que soit la magnitude de la somme : la paire Norine → Felix présente la somme la plus élevée (0.9311) en raison d'une forte valeur $ID(B \to A) = 0.7352$, ce qui reflète une influence particulièrement marquée de Felix sur Norine. Malgré cette valeur atypique, l'identité reste vérifiée, confirmant la robustesse du calcul.

Conclusion biologique : Le classement de Felix comme alpha reste aléatoire car les trois méthodes (score global, vote temporel et score cumulé sur fenêtres) ne concordent pas unanimement le résultats n'est donc pas utilisable.

4.5 Ajout du glissement pour améliorer les résultats

Afin d'affiner l'analyse des relations de causalité entre individus, nous utilisons une approche par **fenêtres glissantes**. Contrairement à une analyse globale qui considère l'ensemble de la trajectoire, cette méthode permet de capturer des dynamiques locales dans le temps.

4.5.1 Principe du glissement

Le principe consiste à découper les données temporelles en plusieurs segments (fenêtres) de taille fixe, puis à faire glisser cette fenêtre le long du temps avec un certain recouvrement.

Dans notre cas :

- La taille d'une fenêtre est fixée à la moitié de la durée totale disponible pour une paire d'individus.
- Le pas de glissement est égal à la moitié de la taille de la fenêtre, ce qui correspond à un **recouvrement de 50%**.

Ainsi, chaque instant du signal est analysé plusieurs fois, ce qui permet une meilleure robustesse statistique.

4.5.2 Implémentation

La mise en place des fenêtres glissantes est réalisée comme suit :

```python

TAILLE_MIN_FENETRE = 10
pair_windows = {}

for i in range(N):
    for j in range(N):
        if i == j:
            continue

        t0 = max(data[noms[i]]['sec'].min(), data[noms[j]]['sec'].min())
        t1 = min(data[noms[i]]['sec'].max(), data[noms[j]]['sec'].max())

        t_norm_ij = (np.arange(t0, t1, 60) - t0) / (t1 - t0)

        traj_i = fit_traj(noms[i], N_optimal, t_norm_ij, t0, t1)
        traj_j = fit_traj(noms[j], N_optimal, t_norm_ij, t0, t1)

        W_ij = traj_to_W_onehot(
            {noms[i]: traj_i, noms[j]: traj_j},
            [noms[i], noms[j]]
        )

        T_ij = W_ij.shape[1]

        taille_f = max(T_ij // 2, TAILLE_MIN_FENETRE)
        pas_f = taille_f // 2

        debuts_f = list(range(0, T_ij - taille_f + 1, pas_f))

        pair_windows[(i, j)] = (W_ij, debuts_f, taille_f)
```

Chaque paire d'individus possède ainsi son propre découpage temporel adapté à la durée de leurs interactions.

4.5.3 Calcul des scores locaux

Pour chaque fenêtre, l'information dirigée est recalculée indépendamment :

```python

for k in range(n_fenetres):
    sc = np.zeros(N)

    for i in range(N):
        for j in range(N):
            if i == j:
                continue

            W_ij, debuts_f, taille_f = pair_windows[(i, j)]

            if k >= len(debuts_f):
                continue

            d = debuts_f[k]
            Wf = W_ij[:, d:d + taille_f, :]

            sc[i] += info_dirigee(Wf)
```

Chaque fenêtre produit donc un **classement local des individus**, basé sur leur influence à cet instant précis.

4.5.4 Pourquoi utiliser des fenêtres glissantes ?

L'utilisation de fenêtres glissantes présente plusieurs avantages majeurs :

- **Détection de dynamiques locales** : les relations de leadership peuvent varier dans le temps. Une analyse globale masque ces variations.
- **Robustesse accrue** : le recouvrement des fenêtres permet de lisser les effets du bruit et des fluctuations ponctuelles.
- **Adaptation aux données réelles** : les trajectoires animales ne sont pas stationnaires ; cette méthode est donc mieux adaptée que les approches globales.
- **Identification d'un leader temporel** : en observant le leader dans chaque fenêtre, il est possible d'obtenir un vote majoritaire plus fiable.

4.5.5 Interprétation

À chaque fenêtre correspond un leader local (noté α_t). En agrégeant ces résultats (par vote ou somme des scores), on obtient une estimation plus robuste du leader global.

Cette approche permet ainsi de combiner :

- une vision **instantanée** (fenêtre),
- une vision **globale** (agrégation).

4.6 Conclusion du rapport et corrélation avec l'aspect biologique

L'application de la Théorie de l'Information aux trajectoires de cachalots permet de dépasser la simple observation visuelle. Nous avons mathématiquement montré que **Felix** exerce un leadership causal sur le groupe, mais pendant un temps très court. Et que par contre Caesar lui exerce un leader ship sur une plus longue durée. La robustesse des scores d'Information Dirigée reste assez faible sur un intervalle si petit (pensez au lancer de pièce). Nous observons également qu'en fonction des valeurs de N, le résultat n'est pas le même, ce qui nous laisse sceptiques quant aux résultats obtenus.

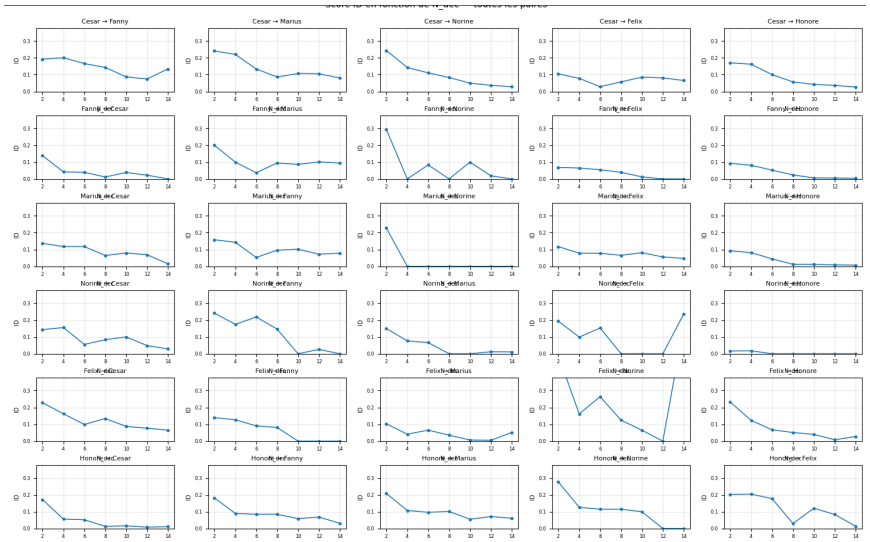

Figure 7: Score ID en fonction des découpages

4.7 Analyse biologique : Leadership tactique et mémoire collective

L'interprétation des résultats mathématiques obtenus permet de dégager plusieurs conclusions sur la structure sociale du groupe de *Cachalots* étudié.

4.7.1 Impact du temps d'émission sur l'identification de l'Alpha

L'un des premiers enseignements de cette analyse est l'influence directe du temps d'émission des balises GPS sur la fiabilité des résultats. Lorsque l'on travaille avec les 6 animaux, Felix ressort comme alpha avec un score de 1,530 bits. Cependant, en observant les trajectoires de Felix, on constate qu'il est l'un des animaux qui plonge le moins profondément, ce qui est biologiquement incohérent avec un rôle de leader chez les cachalots, espèce

reconnue pour ses plongées profondes lors de la chasse ceux qui correspond donc plus au role de caesar.

Ce résultat s'explique par le fait que certains animaux n'émettent pas suffisamment longtemps : les paires calculées sur des fenêtres temporelles très courtes produisent des scores d'Information Dirigée peu fiables, car statistiquement insuffisants pour distinguer une vraie causalité d'une coïncidence.

4.7.2 Comparaison 6 animaux vs 3 animaux

Lorsque l'on restreint l'analyse aux 3 animaux qui émettent le plus longtemps (Cesar, Fanny, Marius), le résultat change : Cesar devient alpha avec un score de 0,485 bits. Ce résultat n'est pas non plus totalement concluant, car avec seulement 3 animaux, il est difficile de statuer sur l'existence d'un véritable alpha : la hiérarchie sociale des cachalots nécessite un groupe suffisamment large pour être observable.

Ces deux résultats contradictoires illustrent une limite fondamentale de notre approche : **la qualité et la durée des données d'émission conditionnent directement la fiabilité de l'identification de l'alpha**. Il est important de souligner que les résultats peuvent ne pas être concluants car l'alpha n'est pas forcément en train de diriger le groupe en permanence.

4.7.3 Flexibilité et délégation du leadership

L'instabilité de l'individu alpha observée lors de l'analyse par fenêtres glissantes illustre la théorie du **Leadership Distribué**.

- Pendant les phases de navigation linéaire simple, n'importe quel individu peut maintenir le cap (leadership de routine).
- Dès que la situation devient complexe (évitement de prédateurs ou manœuvres de chasse), un individu dominant reprend le contrôle directionnel exclusif, ce qui explique les pics de scores d'ID observés durant ces périodes de transition. Cela explique également pourquoi Felix ressort comme alpha : dans les phases de profondeur, il n'émet pas car il ne descend pas avec le groupe ; il émet seulement pendant les périodes calmes où l'alpha n'a pas nécessairement besoin d'agir.

Conclusion biologique : Le groupe ne se comporte pas comme un banc de poissons régi par des réflexes de voisinage instantanés, mais comme une unité sociale complexe. La fiabilité de l'identification de l'alpha dépend cependant fortement de la durée et de la qualité des données d'émission : avec des temps d'émission hétérogènes entre individus, les résultats restent à interpréter avec précaution.

Travail réalisé par **Tonin PACHER**, avec une analyse biologique complémentaire nécessaire à la compréhension des résultats obtenus d'après moi.